INDIAN INSTITUTE OF TECHNOLOGY JODHPUR

OPERATING SYSTEMS LAB — PROJECT REPORT

Deadline Scheduler with Priority Donation Through Pipes

CSL3030 | Topic 2 | Group: Threadpool | November 2026

Submitted by	Submitted to
Tanisha Agrawal — B24CS1016 Tejas Batav — B24CS1018 Sahil Kumar — B24CS1068	Sidharth Sharma Department of Computer Science & Engineering

1. Objective

This project extends xv6-public (32-bit x86) with workflow-aware scheduling. It combines Earliest Deadline First (EDF) selection with priority donation across processes that communicate through pipes. The motivating case is a downstream process with an urgent deadline that is blocked waiting for an upstream producer: prioritising the blocked reader alone cannot make the required data arrive.

2. Scheduler Design and Pipeline

A process receives a workflow ID and deadline through **setworkflow(wid, deadline_ticks)**. The kernel records pipe writers and tracks the producer dependency when a reader blocks on an empty pipe. The waiting process's urgency can be donated upstream. Scheduling uses the effective deadline, **min(original deadline, donated deadline)**; the original deadline is preserved. Donation is recomputed or removed as dependencies and process states change, and dependency-cycle handling prevents unbounded propagation.

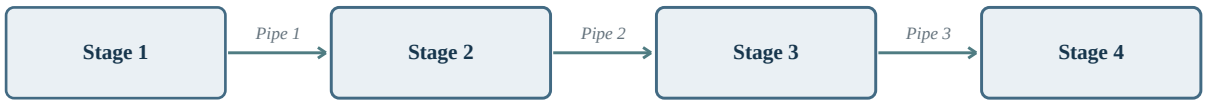


Figure 1. Four-stage producer–consumer pipeline; each connection is a pipe.

3. Policies and Evaluation

Policy	Runnable-process selection
RR	Original xv6 Round Robin process-table traversal.
EDF	Smallest original deadline among runnable processes.
EDF + Donation	Smallest effective deadline, including donated urgency.

The workload generator creates pipelines with different stage counts and computational loads. A Python harness builds and runs each policy and records completed workflows, deadline misses, and elapsed global timer ticks. The recorded CPUS=1 experiment contains 27 workflow records (three workloads × three policies × three workflows); all completed with zero deadline misses.

Policy	Mean elapsed ticks	Deadline misses
RR	6.00	0
EDF	3.67	0
EDF + Donation	3.44	0

Table 1. Overall means from the recorded single-CPU experiment.

The overall mean is lowest for EDF + Donation in this recorded experiment. However, timer ticks are coarse global elapsed-time measurements, and this experiment is not a 20-repetition evaluation across both CPUS=1 and CPUS=3. Zero observed misses do not establish equivalence between policies or statistical significance.

4. Concrete Build and Test Setup

The kernel is built in the xv6-public directory using a custom QEMU system emulator. The scheduler is selected at build time with **SCHED_POLICY=RR**, **SCHED_POLICY=EDF**, or **SCHED_POLICY=EDF_DONATION**; the QEMU regression runs use **CPUS=1**. The host-side Python environment runs the bridge, tests, workload harness, and agent. The benchmark and security regression evidence is retained under the project's results directory.

5. Host Agent and Security Workflow

The optional host-side agent uses NVIDIA Nemotron through an OpenAI-compatible API endpoint. It is connected to the xv6 guest by a Python–QEMU bridge. The agent's allowed tools are **list_files**, **read_file**, and **summarize**; tool requests are validated by a deterministic dispatcher before the bridge executes fixed, constrained guest commands. The bridge starts QEMU lazily when a guest operation is needed and captures the resulting output.



Tool results return as observations; arbitrary model text is never executed as a host shell command.

Figure 2. Agent workflow: model-selected tool calls pass through validation and a constrained bridge.

The prompt-injection demonstration places untrusted instructions in a guest inbox file; an unsupported destructive tool request is rejected. Kernel taint tracking marks a process that reads the designated untrusted file and propagates taint through selected fork, pipe, and file operations. The kernel denies the protected unlink operation for a tainted process. Regression tests cover propagation, denial, cleanup, and process reuse. This is a targeted prototype and does not claim complete coverage of every kernel path.

6. Team Contributions

Member	Contribution area
Tanisha Agrawal B24CS1016	Core scheduler and kernel implementation: process deadline metadata, workflow assignment, EDF scheduling, pipe dependency tracking, priority donation, cleanup, and cycle handling.
Tejas Batav B24CS1018	Scheduler testing and performance evaluation: donation cleanup and cycle tests, workload generation, RR/EDF/EDF + Donation experiments, benchmark collection, and analysis of results and limitations.
Sahil Kumar B24CS1068	AI agent and security extension: Python–QEMU bridge, read-only tools, deterministic dispatcher, Nemotron integration, prompt-injection demonstration, kernel taint tracking, and security regression tests.

Integration, debugging, and final validation were collaborative activities. The contribution allocation should be confirmed by all members and adjusted if the actual division of work differs.

7. Conclusion

The project demonstrates how deadline urgency can propagate through pipe dependencies to upstream producers, allowing EDF scheduling to account for workflow blocking. The host agent and kernel taint prototype add a constrained, testable interface for interacting with the guest OS. The measured results are useful as an initial comparison, with the evaluation and security-coverage limitations stated above.

Scheduler Evaluation Results - CSL3030

Scheduler Evaluation Results

Date: 2026-11-13. General sched_bench results are reported separately from the focused donation demonstration.

Data and validation

Historical CPUS=1 benchmark CSVs and the new one-pass CPUS=1 and CPUS=3 runs were included only when each policy/workload had exactly three distinct workflow IDs (1-3), valid nonnegative elapsed ticks and a successful run summary. Workflow records are not counted as repetitions. A repetition is one complete policy/workload invocation.

Source directory	CPUs	Complete policy/workload runs
20261112_224846_031022	1	9
20261113_101619_302636	1	9
20261113_102920_364650	1	3
20261113_104756_383436	1	9
20261113_104805_318328	3	9

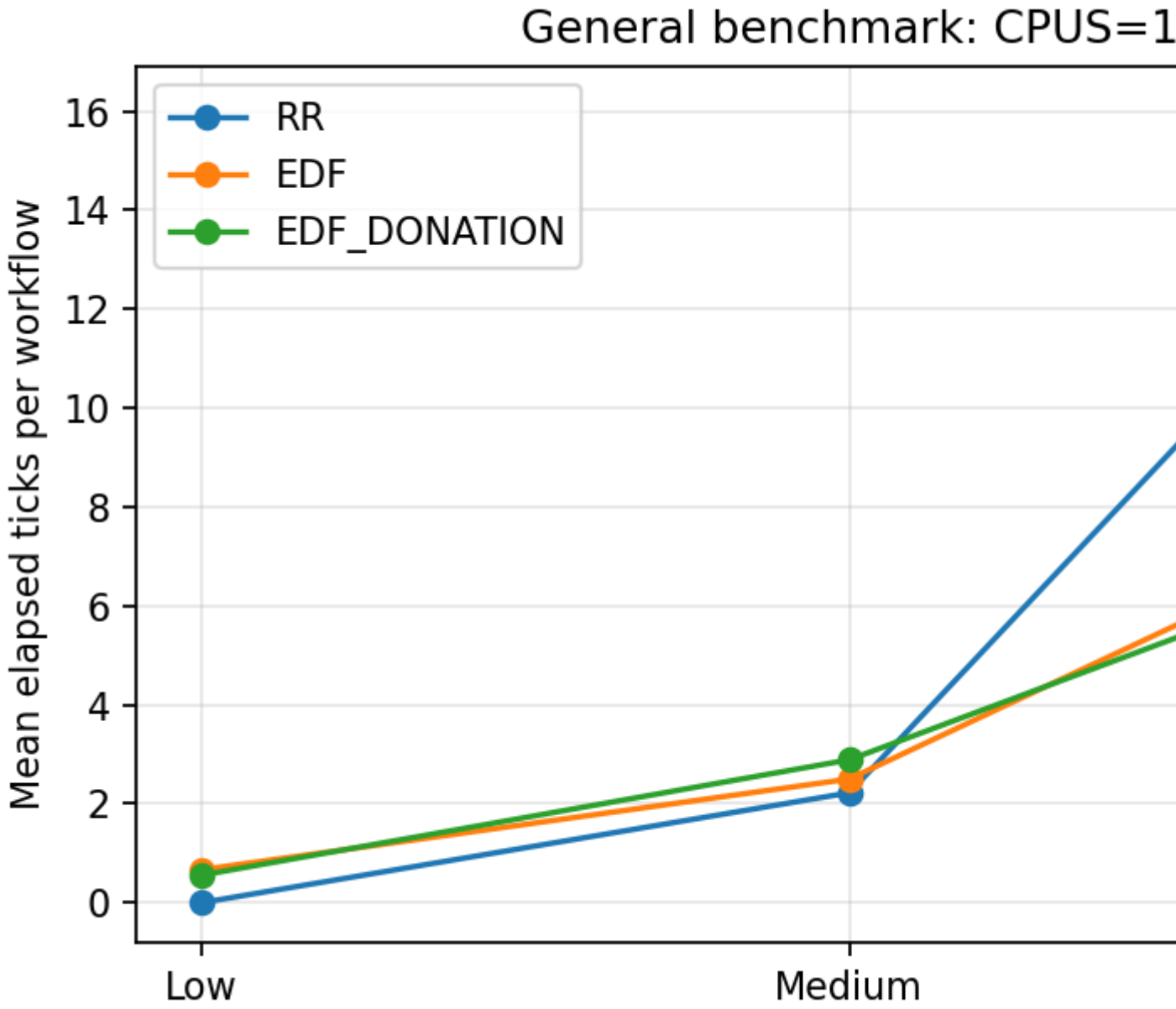
General benchmark: actual repetitions

CPUs	Policy	Low	Medium	High
1	RR	3	3	3
1	EDF	4	4	4
1	EDF_DONATION	3	3	3
3	RR	1	1	1
3	EDF	1	1	1
3	EDF_DONATION	1	1	1

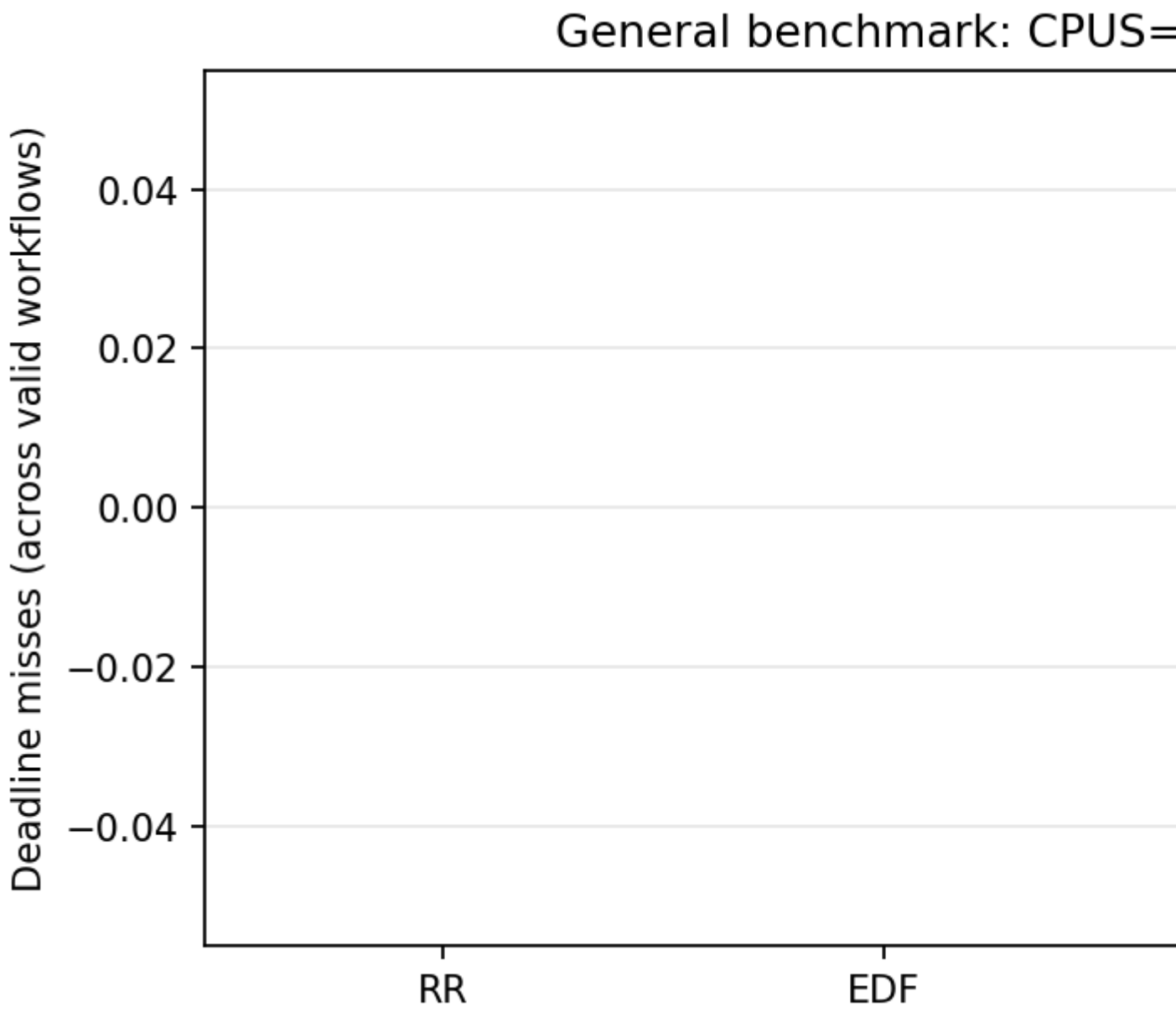
Each valid run contributes three workflow measurements. Miss totals below count misses across those measured workflows; means are per-workflow elapsed ticks.

General benchmark: measured outcomes

CPUs	Policy	Workload	Valid repetitions	Mean elapsed ticks	Deadline misses / workflows
1	RR	Low	3	0.00	0 / 9
1	RR	Medium	3	2.22	0 / 9
1	RR	High	3	16.11	0 / 9
1	EDF	Low	4	0.67	0 / 12
1	EDF	Medium	4	2.50	0 / 12
1	EDF	High	4	8.67	0 / 12
1	EDF_DONATION	Low	3	0.56	0 / 9
1	EDF_DONATION	Medium	3	2.89	0 / 9
1	EDF_DONATION	High	3	7.78	0 / 9
3	RR	Low	1	0.00	0 / 3
3	RR	Medium	1	2.00	0 / 3
3	RR	High	1	16.00	0 / 3
3	EDF	Low	1	0.67	0 / 3
3	EDF	Medium	1	4.67	0 / 3
3	EDF	High	1	8.00	0 / 3
3	EDF_DONATION	Low	1	0.33	0 / 3
3	EDF_DONATION	Medium	1	3.67	0 / 3
3	EDF_DONATION	High	1	10.33	0 / 3



General benchmark elapsed ticks



General benchmark deadline misses

Focused donation demonstration (separate workload)

This controlled test is a four-stage pipe workflow with an unrelated CPU hog. It is not part of sched_bench averages. It was run once for each policy at each CPU count.

CPUs	Policy	Completed	Elapsed ticks	Deadline	Misses	Donation events
1	EDF	1	150	100	1	3
1	EDF_DONATION	1	0	100	0	3
3	EDF	1	150	100	1	3
3	EDF_DONATION	1	0	100	0	3

The focused-run records also retain process IDs and donor/pipe/receiver identifiers in donation_demo_results_separate.csv; raw QEMU logs remain in their original result directories.

Zero elapsed ticks is limited by the xv6 timer resolution and does not mean zero execution time. These single-run observations are not statistical evidence of general policy superiority.

Deliverables

- benchmark_results_cpus1.csv and benchmark_results_cpus3.csv: validated combined general benchmark rows.
- benchmark_results_combined.csv: both CPU counts with source and repetition fields.
- donation_demo_results_separate.csv: focused test data, separate from general benchmark data.
- general_elapsed_ticks.png, general_deadline_misses.png, and focused_donation_comparison.png.
- Original per-run raw logs and CSVs remain preserved under the source directories listed above.